

MACHINE LEARNING

DEEP LEARNING: CONVOLUTIONAL NEURAL NETWORKS

Sebastian Engelke

MASTER IN BUSINESS ANALYTICS



**UNIVERSITÉ
DE GENÈVE**

Image recognition: CIFAR-10

- ▶ A benchmark data set for **image recognition** is **CIFAR-10**:
<https://www.cs.toronto.edu/~kriz/cifar.html>
- ▶ It consists of 60000 32×32 RGB colour images with 10 different classes

$$\mathcal{G} = \{\text{airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck}\}$$

- ▶ Each image has **three channels** (red, green, blue), each consisting of the intensities of that color in each pixel of that image.
- ▶ The predictor space has thus **dimension** $p = 32 \times 32 \times 3 = 1024$.

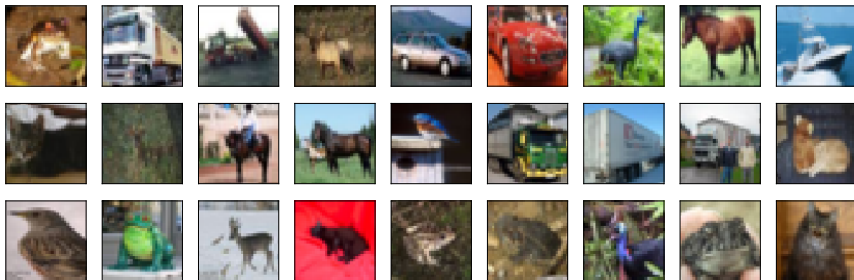
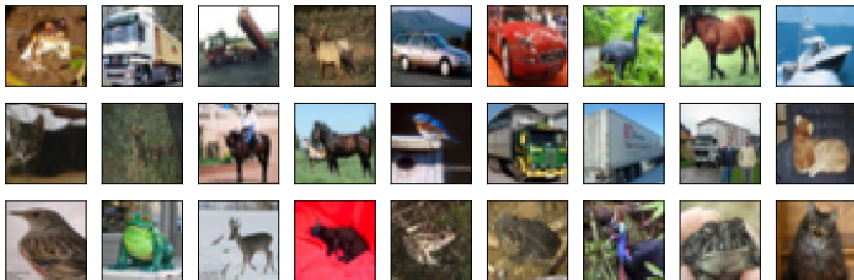


Image recognition: CIFAR-10

- ▶ A benchmark data set for **image recognition** is **CIFAR-10**:
<https://www.cs.toronto.edu/~kriz/cifar.html>
- ▶ It consists of 60000 32×32 RGB colour images with 10 different classes

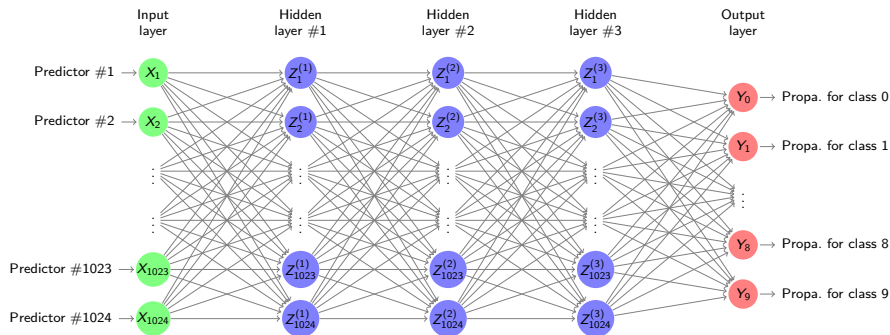
$$\mathcal{G} = \{\text{airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck}\}$$

- ▶ Each image has **three channels** (red, green, blue), each consisting of the intensities of that color in each pixel of that image.
- ▶ The predictor space has thus **dimension** $p = 32 \times 32 \times 3 = 1024$.
- ▶ State-of-the-art methods have prediction accuracy above 95%:
<https://www.kaggle.com/c/cifar-10/leaderboard>



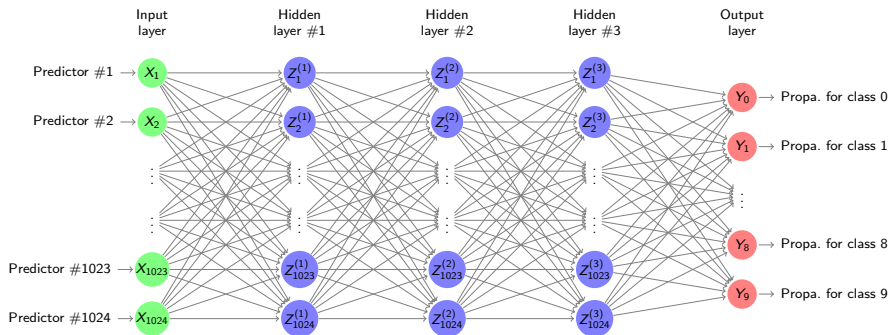
A dense neural network

- For the **CIFAR-10** data set, the number of predictors is $p = 1024$.
- A **dense neural network** is one where **all neurons** are connected with **individual weights**.



A dense neural network

- ▶ For the **CIFAR-10** data set, the number of predictors is $p = 1024$.
- ▶ A **dense neural network** is one where **all neurons** are connected with **individual weights**.
- ▶ For this data set, **each layer** in a dense network has about **1 million parameters**!
- ▶ Fitting so many parameters is **computationally very expensive**.
- ▶ Moreover, with only 60000 training samples, heavy **overfitting will occur**.



Spatial structures in image data

- ▶ We have already seen different methods of regularization that aim at avoiding overfitting.
- ▶ If **domain knowledge** is available, a very effective way of regularization is to **decrease** the number of parameters in the network **architecture**.



Spatial structures in image data

- ▶ We have already seen different methods of regularization that aim at avoiding overfitting.
- ▶ If **domain knowledge** is available, a very effective way of regularization is to **decrease** the number of parameters in the network **architecture**.
- ▶ This can be done by **parameter sharing**.
- ▶ For image data, there is a natural **spatial structure** underlying the predictors: two **neighboring pixels** are usually more **similar** than pixels far apart.



Spatial structures in image data

- ▶ We have already seen different methods of regularization that aim at avoiding overfitting.
- ▶ If **domain knowledge** is available, a very effective way of regularization is to **decrease** the number of parameters in the network **architecture**.
- ▶ This can be done by **parameter sharing**.
- ▶ For image data, there is a natural **spatial structure** underlying the predictors: two **neighboring pixels** are usually more **similar** than pixels far apart.
- ▶ **Convolutional neural networks** incorporate this intuition into the network architecture.



Convolutional neural networks

- ▶ Convolutional neural networks are **deep neural networks** with many hidden layers.
- ▶ The hidden layers can be of **different types**, meaning that different operations or parameter sharings are used.
- ▶ We already know one type of hidden layer, namely the **dense/fully connected layer** whose neurons are **connected to all neurons** of the previous layer (either input or hidden).
- ▶ There are **many other layer types** and we will study some of the most important.

Convolutional neural networks

- ▶ Convolutional neural networks are **deep neural networks** with many hidden layers.
- ▶ The hidden layers can be of **different types**, meaning that different operations or parameter sharings are used.
- ▶ We already know one type of hidden layer, namely the **dense/fully connected layer** whose neurons are **connected to all neurons** of the previous layer (either input or hidden).
- ▶ There are **many other layer types** and we will study some of the most important.
- ▶ A **convolutional layer** tries to **extract local features** of images in three stages:
 - a) the convolution stage;
 - b) the activation stage;
 - c) the pooling stage.

The convolutional stage

- The **first stage** is the **convolution operation** that linearly combines the inputs of its **receptive field** (e.g., 3×3 pixels) to a single score.

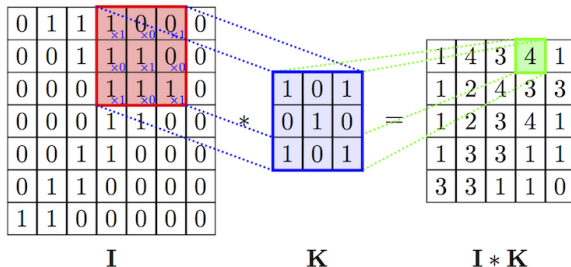


Image from <https://cambridgespark.com/content/tutorials/convolutional-neural-networks-with-keras/index.html>

The convolutional stage

- ▶ The **first stage** is the **convolution operation** that linearly combines the inputs of its **receptive field** (e.g., 3×3 pixels) to a single score.
- ▶ The **weights** for this linear combination are called a **filter**, and different filters extract different features (horizontal lines, vertical lines, bright spots, etc.).
- ▶ The receptive field **slides** through the whole image and thus producing a whole grid of neurons as output.

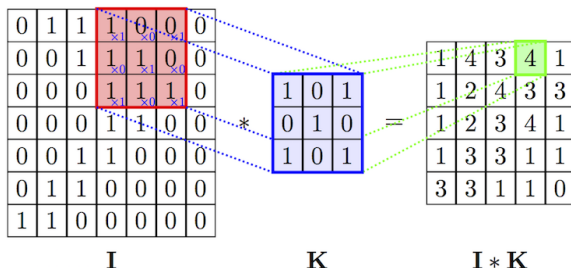


Image from <https://cambridgespark.com/content/tutorials/convolutional-neural-networks-with-keras/index.html>

The convolutional stage

- ▶ The **first stage** is the **convolution operation** that linearly combines the inputs of its **receptive field** (e.g., 3×3 pixels) to a single score.
- ▶ The **weights** for this linear combination are called a **filter**, and different filters extract different features (horizontal lines, vertical lines, bright spots, etc.).
- ▶ The receptive field **slides** through the whole image and thus producing a whole grid of neurons as output.
- ▶ Below the input image is called I , the filter is K and the **convoluted image** $I * K$ is the output of this layer.
- ▶ The filter can be fixed, but usually its weights are **learned** during the fitting.
- ▶ The only parameters of this layer are the **weights of the filter** and a bias.

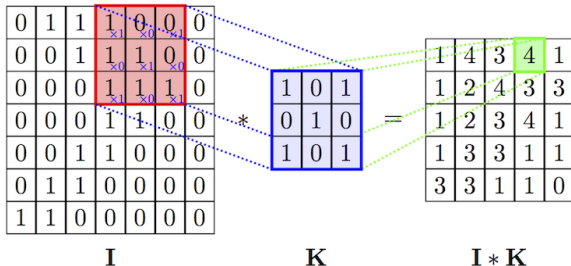


Image from <https://cambridgespark.com/content/tutorials/convolutional-neural-networks-with-keras/index.html>

The convolution stage

- Convolutional layers have several **hyper parameters**, e.g., the **width** and the **height** of the filter.

The convolution stage

- ▶ Convolutional layers have several **hyper parameters**, e.g., the **width** and the **height** of the filter.
- ▶ There is also the **depth** of the layer, that is, the number of **different filters** applied in the same step.

The convolution stage

- ▶ Convolutional layers have several **hyper parameters**, e.g., the **width** and the **height** of the filter.
- ▶ There is also the **depth** of the layer, that is, the number of **different filters** applied in the same step.
- ▶ The **stride** specifies by how much we **shift the filter** to obtain the next pixel of the output.

The convolution stage

- ▶ Convolutional layers have several **hyper parameters**, e.g., the **width** and the **height** of the filter.
- ▶ There is also the **depth** of the layer, that is, the number of **different filters** applied in the same step.
- ▶ The **stride** specifies by how much we **shift the filter** to obtain the next pixel of the output.
- ▶ The output image may have **smaller size** than the input image.
- ▶ **Zero-padding** (also called “same” padding) might be used to **keep same sizes** of input and output; in this case the image is sufficiently padded with **zeros** at the edges.

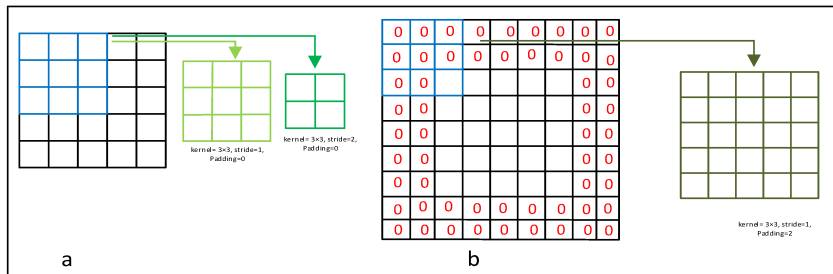


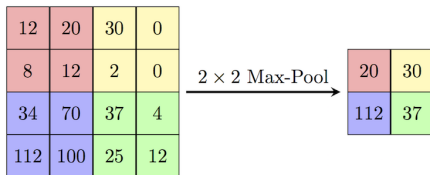
Image from <http://www.mdpi.com/2078-2489/9/1/19>

The detector and pooling stages

- ▶ The **second** stage of the convolutional layer is the **detector stage** where a non-linearity is applied to each neuron, usually the **ReLU**.
- ▶ A neuron is **active** and the feature is **present** in that part of the image only if the linear score of the filter is high enough.
- ▶ This stage does not have any parameters when the non-linearity is fixed.

The detector and pooling stages

- ▶ The **second** stage of the convolutional layer is the **detector stage** where a non-linearity is applied to each neuron, usually the **ReLU**.
- ▶ A neuron is **active** and the feature is **present** in that part of the image only if the linear score of the filter is high enough.
- ▶ This stage does not have any parameters when the non-linearity is fixed.
- ▶ The third stage is **pooling** which is a form of **downsampling**.
- ▶ Typically, **max-pooling** is applied to aggregate chunks of the image into a single value by taking the **maximum activation** of the convolution output in that chunk (often 2×2).



The detector and pooling stages

- ▶ The **second** stage of the convolutional layer is the **detector stage** where a non-linearity is applied to each neuron, usually the **ReLU**.
- ▶ A neuron is **active** and the feature is **present** in that part of the image only if the linear score of the filter is high enough.
- ▶ This stage does not have any parameters when the non-linearity is fixed.
- ▶ The third stage is **pooling** which is a form of **downsampling**.
- ▶ Typically, **max-pooling** is applied to aggregate chunks of the image into a single value by taking the **maximum activation** of the convolution output in that chunk (often 2×2).

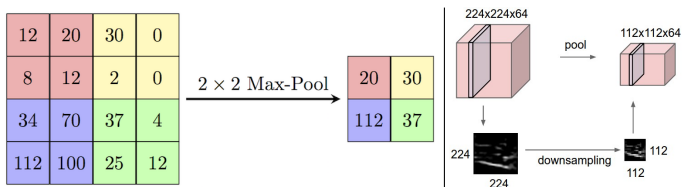


Image from <https://cambridgespark.com/content/tutorials/convolutional-neural-networks-with-keras/index.html>

The convolutional layer

- ▶ The **convolutional layer** are the three stages together

convolution operation $\rightarrow$ ReLU $\rightarrow$ pooling,

(sometimes only the convolutional operation is called a convolutional layer).

- ▶ There may be **several of these layers** in a deep neural network, sometimes with several convolutions plus ReLU operations before pooling.

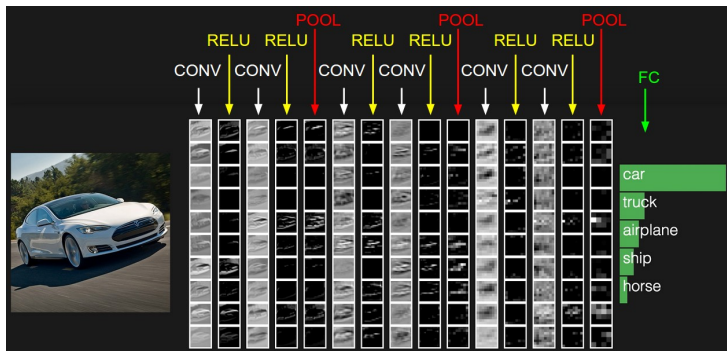


Image from <http://cs231n.github.io/convolutional-networks/>

Convolutional layers



Filter 1

-1	-1	1	1
-1	-1	1	1
-1	-1	1	1
-1	-1	1	1

Filter 2

1	1	-1	-1
1	1	-1	-1
1	1	-1	-1
1	1	-1	-1

Filter 3

-1	-1	-1	-1
-1	-1	-1	-1
1	1	1	1
1	1	1	1

Filter 4

1	1	1	1
1	1	1	1
-1	-1	-1	-1
-1	-1	-1	-1

Activation Map for Filter 1



Activation Map for Filter 2



Activation Map for Filter 3



Activation Map for Filter 4



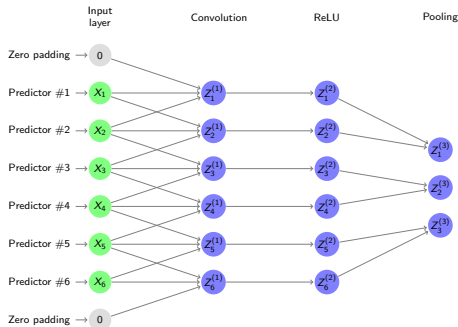
Convolutional layer

- Mathematically a **convolutional layer** is defined as

$$Z_j^{(1)} = b_j^{(1)} + w_j^{(1)\top} X, \quad j = 1, \dots, m_1, \quad Z_j^{(2)} = \max\{0, Z_j^{(1)}\}, \quad j = 1, \dots, m_1$$
$$Z_j^{(3)} = \text{pool}(Z^{(2)})_j, \quad j = 1, \dots, m_3$$

with equal biases $b_1^{(1)} = \dots = b_m^{(1)}$ and weight vectors $w_1^{(1)}, \dots, w_m^{(1)}$ having zeros except for close pixels (defined by the **filter**), and moreover the **weights are shared**, that is, equal in all m vectors, but shifted to match the neighbors.

- The **number of parameters** are therefore only **filter height $\times$ filter width $+ 1$** , in the example below $3 \times 1 + 1 = 4$.



A typical CNN

- ▶ Often more than one filter is used in one **convolutional layer**.
- ▶ The **number of parameters** is $(\text{filter height} \times \text{filter width} + 1) \times \text{filter depth}$, where **filter depth** is the number of filters.
- ▶ In the first convolutional layer below $(5 \times 5 + 1) \times 10 = 260$.

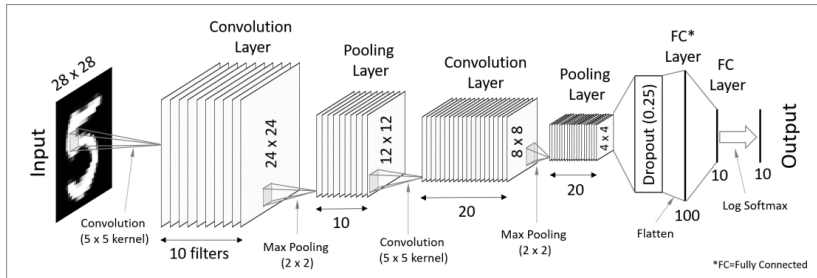


Image from

<https://codetolight.wordpress.com/2017/11/29/getting-started-with-pytorch-for-deep-learning-part-3-neural-network-basics/>

A typical CNN

- ▶ Often more than one filter is used in one **convolutional layer**.
- ▶ The **number of parameters** is $(\text{filter height} \times \text{filter width} + 1) \times \text{filter depth}$, where **filter depth** is the number of filters.
- ▶ In the first convolutional layer below $(5 \times 5 + 1) \times 10 = 260$.
- ▶ Note: A **flattening layer** simply transforms a 2-dimensional layer into a 1-dimensional vector in order to use a **fully connected layer**.

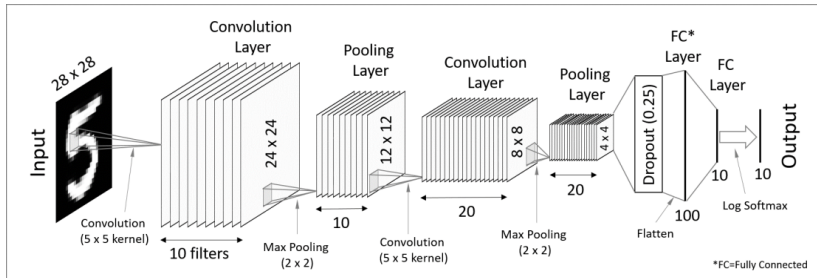


Image from

<https://codetolight.wordpress.com/2017/11/29/getting-started-with-pytorch-for-deep-learning-part-3-neural-network-basics/>

Digit classification with neural networks



► We fit a **fully connected** and a **convolutional neural network**.

- linreg: direct linear regression;
- LDA1: LDA based on the values x_i ;
- LDA2: LDA based on x_i and x_i^2 ;
- QDA: different cov. matrix for each class;
- log: logistic regression (multinomial).
- log-ridge: logistic regression with ridge.
- log-lasso: logistic regression with lasso.
- RF: random forest.
- linear SVM: linear support vector classifier.
- radial SVM: SVM with radial kernel.
- FC NN: fully connected neural network.
- CNN: convolutional neural network.

	Training error	Test error
linreg	7.6%	13.1%
LDA1	6.2%	11.5%
LDA2	3.9%	10.2%
QDA	1.8%	13.5%
log	0.01%	11.1%
log-ridge	4.1%	9.0%
log-lasso	2.7%	8.8%
RF	0%	5.9%
linear SVM	1.5%	6.5%
radial SVM	0.6%	5.4%
FC NN	~ 0%	5.8%
CNN	~ 0%	3.2%

Activations and filters of first layer

- The first layer extract **generic features** of the image.

